Lab 3

Introduction to DDL and DML Operations

Group 2A

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

1 Table Management (DDL)

1.1 Creating a Table

Creating a table defines the structure of your data. You must define columns, data types, and constraints.

General Syntax:

```
CREATE TABLE table_name
(
    attribute1 datatype [ NULL | NOT NULL | UNIQUE ],
    attribute2 datatype [ NULL | NOT NULL | UNIQUE ],
    ...
);
```

1.1.1 Common PostgreSQL Data Types

- `INT`: Standard whole numbers. *Warning: Removes leading zeros (001 becomes 1).*
- `VARCHAR(n)`: Variable-length string. **Use this for IDs with leading zeros** (e.g., Zip Codes, Student IDs like `'00128'`).
- `CHAR(n)`: Fixed-length string. Pads with spaces if input is short.
- `SERIAL`: Auto-incrementing integer (1, 2, 3...). Perfect for internal IDs.
- `NUMERIC(p,s)`: Exact decimals (e.g., Currency).
- `DATE`: Calendar dates.

1.1.2 Example: Constraints

Constraints protect data quality.

```
CREATE TABLE PRODUCT
(
    PID SERIAL PRIMARY KEY,         -- Unique, Auto-increment
    PNAME VARCHAR(50) NOT NULL,     -- Cannot be empty
    PRICE INT,
    CONSTRAINT PRICE_CHECK CHECK (PRICE >= 0) -- No negative prices
);
```

1.2 Dropping Tables

```
DROP TABLE table_name;
```

The Cascade Option: If other objects depend on this table, use `CASCADE` to force deletion.

```
DROP TABLE PRODUCT CASCADE;
```

1.3 Altering Tables

Modify structure without deleting data.

```
-- 1. Add a new column
ALTER TABLE PRODUCT ADD COLUMN STOCK INT;

-- 2. Remove a column
ALTER TABLE PRODUCT DROP COLUMN STOCK;

-- 3. Change data type
ALTER TABLE PRODUCT ALTER COLUMN PRICE TYPE NUMERIC(10,2);

-- 4. Rename column
ALTER TABLE PRODUCT RENAME COLUMN PNAME TO PRODUCT_NAME;
```

2 Data Manipulation (DML)

2.1 Inserting Records

Named Notation (Recommended): Specifying columns is safer than relying on column order.

```
INSERT INTO PRODUCT(PID, PNAME, PRICE) VALUES (DEFAULT, 'Pen', 15);
```

2.2 Updating Data

Warning: Always use a `WHERE` clause, otherwise all rows are updated.

```
UPDATE PRODUCT
SET PRICE = 250
WHERE PID = 1;
```

2.3 Deleting Data

Warning: Without a `WHERE` clause, the table becomes empty.

```
DELETE FROM PRODUCT
WHERE PRICE = 0;
```

2.4 Retrieval (SELECT) and Filtering

The core of SQL is querying data.

Basic Filtering:

```
SELECT * FROM PRODUCT WHERE PRICE > 50;
```

2.4.1 Logical Operators (AND, OR)

To answer complex questions, we combine conditions.

1. **AND Operator:** Both conditions must be true.

```
-- Find products that are expensive AND in a specific category
SELECT * FROM PRODUCT
WHERE PRICE > 100 AND CATEGORY = 'Electronics';
```

2. **OR Operator:** At least one condition must be true.

```
-- Find products that are either Blue OR Red
SELECT * FROM PRODUCT
WHERE COLOR = 'Blue' OR COLOR = 'Red';
```

2.5 Working with Dates (DATE)

The DATE type stores a calendar date. Write a date as a string in the format 'YYYY-MM-DD'; PostgreSQL converts it to a DATE automatically.

```
CREATE TABLE ORDERS
(
    OID INT PRIMARY KEY,
    ORDER_DATE DATE NOT NULL,
    AMOUNT INT
);

INSERT INTO ORDERS (OID, ORDER_DATE, AMOUNT) VALUES
(1, '2026-03-15', 500),
(2, '2026-01-07', 1200),
(3, '2026-02-20', 800);
```

Dates can be compared just like numbers. An earlier date is "smaller":

```
-- Orders placed after 1 February 2026
SELECT * FROM ORDERS WHERE ORDER_DATE > '2026-02-01';

-- Orders placed in February 2026
SELECT * FROM ORDERS
WHERE ORDER_DATE >= '2026-02-01' AND ORDER_DATE < '2026-03-01';

-- Today's date
SELECT CURRENT_DATE;
```

2.6 Sorting Results (ORDER BY)

Rows come back in no guaranteed order. Use ORDER BY to sort them. ASC (ascending, the default) puts the smallest value first; DESC (descending) puts the largest first. For dates, ascending means oldest first.

```
-- Oldest order first
SELECT * FROM ORDERS ORDER BY ORDER_DATE ASC;

-- Newest order first
SELECT * FROM ORDERS ORDER BY ORDER_DATE DESC;

-- Filter first, then sort: big orders, largest amount first
```

```
SELECT * FROM ORDERS
WHERE AMOUNT > 600
ORDER BY AMOUNT DESC;
```

`ORDER BY` always comes after `WHERE`.

Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

   1. Connect to Your Database:

   Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

   2. Verify the Connection:

   Verify that you are connected to the correct database with the following command:

```
\conninfo
```

   Expected output:

```
You are connected to database "cse4308_12345" as user "student_12345".
```

   3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

   Load the SQL file that sets up the tables and inserts initial data.

   1. Load the SQL File:

   Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_03_2a_setup.sql
```

   This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

   After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

   1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

   2. Check Tables:

   Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L3_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

3 Lab Task

The `GPU_SALE` table:

- `TXN_ID` (INT, Transaction ID, Primary Key)
- `SALE_DATE` (DATE, Must not be NULL)
- `PRICE` (INT, Sold price in USD)
- `MODEL` (VARCHAR(50))

TXN_ID	SALE_DATE	PRICE	MODEL
1001	2026-01-14	1850	RTX 4090
1002	2026-01-20	999	RTX 4080
1003	2026-02-03	920	RX 7900 XTX
1004	2025-12-28	1720	RTX 4090
1005	2026-02-11	0	RTX 4080
1006	2026-03-05	549	RTX 4070
1007	2026-01-02	1990	RTX 4090
1008	2026-02-25	879	RX 7900 XTX
1009	2026-03-18	1800	RTX 4090
1010	2026-03-01	0	RX 7900 XTX
1011	2025-11-30	1060	RTX 4080
1012	2026-02-14	2100	RTX 4090
1013	2026-03-22	529	RTX 4070

Create the `GPU_SALE` table, then insert the data as shown in the `GPU_SALE` table by running `lab_03_2a_setup.sql`. Then perform the following queries on the `GPU_SALE` table:

1. Display all records of the `GPU_SALE` table.
2. Find the transaction IDs and sale dates of RTX 4090 cards that sold for more than 1,800.
3. Find the transactions of either the RTX 4080 or the RX 7900 XTX.
4. Show all transactions of the RTX 4090, sorted by sale date from oldest to newest.
5. Delete cancelled transactions (`PRICE = 0`).
6. Drop the `GPU_SALE` table.